

3.6.2 Multiple Threads

Running potentially long operations on separate thread is better implementation. The operating system slices the available processing time among multiple tasks running on multiple threads so that no task dominates execution. It may hence appear that the processes are occurring simultaneously. Hence loading the network call for updating on a separate thread will not block the UI thread, and the application will be much more responsive.

3.7 Accomplishing Multi-threading

Thread class in Java allows for multi-threading operations. Regular Java features can be used to put the network call in the background. A standard Java thread class doesn't allow one thread to update elements of the main UI thread, where we would need to synchronise with the current state. Android provides AsyncTask utility designed for this purpose.

3.7.1 AsyncTask

This Android mechanism helps to handle long operations that need to report to the UI thread. Create a new AsyncTask subclass and implement the following methods:

- `doInBackground()` fills in for background activity

- `onProgressUpdate()` instructs the follow-up activity upon certain progress

- `onPostExecute()` defines the action upon process completion

Following will be your modified code in `StatusActivity.java` with implementation of an asynchronous thread:

```
1 package com.thinkdigit.envy;
2 import winterwell.jtwitter.Twitter;
3 import winterwell.jtwitter.TwitterException;
4 import android.app.Activity;
5 import android.os.AsyncTask;
6 import android.os.Bundle;
7 import android.util.Log;
8 import android.view.View;
9 import android.view.View.OnClickListener;
10 import android.widget.Button;
11 import android.widget.EditText;
12 import android.widget.Toast;
13 public class StatusActivity extends Activity implements
    OnClickListener{
```